

DOCUMENTACIÓN

VISITAS VIRTUALES



Proyecto: Visitas Virtuales

Versión: 1.0 — Documento unificado para entrega final

Equipo de proyecto: Eugenio Rodríguez | Javier Domínguez | María Gómez | Javier Vaquero
| Jose Luis Ruiz

Documentación: Jose Luis Ruiz

Índice

1. Documentación General del Proyecto

1.1 Propósito y visión

1.2 Alcance inicial vs. final

1.3 Cronograma de desarrollo

1.4 Hitos importantes

1.5 Equipo de proyecto y tareas (Basada en la plataforma Jira)

2. Documentación Funcional y Técnica

2.1 Backend

2.1.1 Controller

2.1.2 Service

2.1.3 Repository

2.1.4 Model

2.1.5 Security y Config

2.1.6 Visitasvirtuales_backend y Resources

2.1.7 java/com/ies/tour/visitasvirtuales_backend

2.1.8 Test

2.1.9 Dto

2.1.10 Exception

2.1.11 Payload/Response

2.2 Frontend

2.2.1 HTML

2.2.2 JavaScript

2.2.3 CSS

2.2.4 Frontend — Imágenes

2.3 Resumen de jerarquía y diagrama de interacciones

2.3.1 Jerarquía del Frontend

2.3.2 Jerarquía del Backend (src)

2.3.3 Diagrama de Interacciones Simplificado

2.4 Arquitectura del Sistema

2.5 Diagrama de Componentes (Resumen)

2.6 Estructura del Repositorio y Convenciones de Código

2.7 Configuración del Entorno de Desarrollo

2.8 Servicios (Services)

2.8.1 PuntoDeInteresService / PuntoDeInteresServiceImpl

2.8.2 UsuarioService / UsuarioServiceImpl

2.8.3 CentroService / CentroServiceImpl

2.8.4 VisitaVirtualService / VisitaVirtualServiceImpl

2.9 Repositorio (Repository)

2.9.1 PuntoDeInteresRepository

2.9.2 UsuarioRepository

2.9.3 CentroRepository

2.9.4 VisitaVirtualRepository

2.10 Modelos (Entities)

2.10.1 PuntoDeInteres.java

2.10.2 Centros.java

2.10.3 Usuario.java

2.10.4 RolUsuario.java

2.10.5 VisitaVirtual.java

2.11 Seguridad (Security)

2.11.1 SecurityConfig.java

2.11.2 JwtAuthenticationFilter.java

2.11.3 JwtUtils.java

2.11.4 AuthEntryPointJwt.java

2.11.5 CorsConfiguration.java

2.11.6 AppRoleConfig.java

2.12 DTO

2.12.1 LoginRequest.java

2.12.2 PuntoDeInteresDTO.java

2.12.3 RegisterRequest.java

2.13 Exception

2.13.1 ResourceNotFoundException.java

2.14 Payload/Response

2.14.1 JwtResponse.java

2.15 Buenas prácticas y convenciones

2.16 Relaciones y jerarquía de clases

2.17 Frontend

2.17.1 Estructura de carpetas

2.18 Descripción de los archivos y funcionalidades

2.18.1 index.html

2.18.2 viewadmin.html

2.18.3 centros.html

2.18.4 crearpdis.html

2.18.5 gestionpermisos.html

2.18.6 profilesetting.html

2.18.7 registerform.html

2.18.8 virtualview.html

2.18.9 errorpanel.html

2.19 JavaScript

2.20 CSS

2.21 Jerarquía y flujo

2.22 Relación entre Backend y Frontend

2.23 Archivos .class

2.23.1 Función de los Archivos .class

2.23.2 Organización de los .class en el proyecto

2.23.3 Tipos de archivos .class generados

2.23.4 ¿Por qué son importantes los .class en Spring Boot?

2.23.5 Archivo .class principal

2.23.6 Estructura .class

2.24 Diagrama UML — Modelo del Backend

3. Documentación de Pruebas y Calidad

4. Documentación Legal y de Entrega

4.1 Acta de entrega del proyecto

4.2 Licencias de software utilizadas

4.3 Propiedad intelectual y derechos de uso

4.4 Contratos de mantenimiento o soporte

1. Documentación General del Proyecto

1.1 Propósito y visión

El proyecto **Visitas Virtuales** es una aplicación web diseñada para ofrecer visitas virtuales 360° de centros educativos, integrando información de sus programas académicos y servicios. El software permite a usuarios registrados (alumnos y profesores) navegar por los centros de manera virtual, gestionar puntos de interés y recibir notificaciones.

Objetivos principales:

- Proporcionar un catálogo interactivo de centros educativos con visitas virtuales.
- Facilitar la gestión de información de los centros (admin y usuarios).
- Ofrecer experiencia de usuario fluida y segura, con autenticación y roles diferenciados.

Público objetivo:

- Estudiantes interesados en grados medios y superiores.
- Profesores y personal de administración educativa.
- Visitantes generales interesados en información sobre los centros.

Beneficios esperados:

- Acceso rápido e intuitivo a la información de los centros.
- Experiencia de usuario inmersiva mediante tours virtuales 360°.
- Gestión centralizada de usuarios, permisos y contenidos.
- Plataforma escalable para añadir nuevos centros y servicios.

1.2 Alcance inicial vs. final

Alcance inicial:

- Crear un sistema web con módulos de usuarios, administración de centros y visualización de tours virtuales.
- Implementar autenticación básica y control de roles.
- Desarrollar frontend responsive y backend con base de datos integrada.

Alcance final entregado:

- Frontend completo con páginas de registro, login, perfil de usuario, gestión de PDIs (Puntos de Interés), visualización de tours virtuales y administración de roles.
- Backend funcional con controladores, servicios, repositorios y seguridad JWT.
- Base de datos en Plastic SCM y repositorio de código en GitHub.
- Integración con Unity para despliegue de la aplicación y visualización de contenido virtual.

1.3 Cronograma de desarrollo

- **Fase 1:** Planificación y diseño de arquitectura (2 semanas)
- **Fase 2:** Desarrollo del frontend y componentes UI (2 semanas)
- **Fase 3:** Desarrollo del backend y modelo de datos (2 semanas)
- **Fase 4:** Integración con Unity y BDD en Plastic SCM (2 semanas)
- **Fase 5:** Despliegue, revisión de seguridad y ajustes finales (2 semanas)

1.4 Hitos importantes

- **MVP:** Páginas de login, registro, perfil de usuario y visualización básica de tours virtuales.
- **Revisiones:** Revisión del backend, integración frontend-backend, ajustes de UI/UX.
- **Versiones:**
 - v0.1 — Prototipo inicial con funcionalidades básicas.
 - v0.5 — Backend y frontend integrados.
 - v1.0 — Versión final con todos los módulos funcionales.

1.5 Equipo de proyecto y tareas (Basada en la plataforma Jira)

- Eugenio Rodríguez: Mostrar loading y fallback si WebGL, Implementar panel de administración para gestionar contenidos.
- Javier Domínguez: Integrar iframe o template Unity, Controlar resize y fullscreen, Consumir la API desde Unity WebGL, Diseñar y exponer la API para contenidos dinámicos.
- María Gómez: Creación de repositorio y readme.md en formato markdown, Crear layout responsive.
- Javier Vaquero: API mock: endpoints GET /centros, /centros/{id}/hotspots devolviendo JSON estático.
- José Luis Ruiz: Panel de errores y documentación del proyecto.

2. Documentación Funcional y Técnica

En esta sección se describe **qué hace el sistema**, explicando los módulos, clases y funciones más relevantes de los archivos del **backend** y **frontend**. No se incluye el código completo, solo una descripción detallada de su funcionamiento.

- **README.md**

Este README corresponde al proyecto **VisitaDAVANTE**, una plataforma digital orientada a ofrecer visitas virtuales 360 a centros de formación Davante.

Estructura del README:

El documento está organizado en varios bloques.

Título y Descripción inicial:

VisitaDAVANTE

El README comienza explicando la finalidad del proyecto:

- Es una plataforma que ofrece visitas virtuales 360° a los centros Davante.
- Permite conocer instalaciones, obtener información institucional y acceder a contenido formativo.
- También incluye un área de alumnos para estudiantes matriculados.

2.1 Backend

En esta sección se detallan **las funcionalidades de cada archivo, clase y método**, explicando qué hace y cómo interactúa con el sistema.

Directorios principales:

2.1.1 Controller

Objetivo: Gestionar las solicitudes HTTP entrantes, coordinar servicios y devolver respuestas al frontend.

Archivos y funcionalidades principales:

-Valid.java

Funcionalidad

Esta anotación personalizada simula la etiqueta @Valid del paquete de validación de Spring/Jakarta. En este proyecto se usa para evitar conflictos de dependencias.

Importancia en el proyecto

- Permite anotar DTOs en controladores para validar campos antes de procesarlos.
- **Es clave para señalar intención de validación** en los endpoints.

Métodos / elementos

Esta clase contiene únicamente la declaración de la anotación:

```
public @interface Valid {}
```

Explicación técnica:

- Marca parámetros que deben ser validados.
- No aplica lógica: funciona como marcador semántico dentro del código.

Relaciones

- Usada junto a DTOs: LoginRequest, RegisterRequest, PuntoDeInteresDTO.
- Detectada en controladores del proyecto para validación manual o futura integración con un framework de validación completo.

-PuntoDeInteresController.java

- Gestiona CRUD de Puntos de Interés (PDIs) asociados a los centros.
- Métodos principales:
 - **getAllPdis()** → Devuelve todos los PDIs de la plataforma.
 - **getPdiById(Long id)** → Devuelve un PDI específico según su ID.
 - **createPdi(PuntoDeInteres pdi)** → Crea un nuevo PDI.
 - **updatePdi(Long id, PuntoDeInteres pdi)** → Modifica un PDI existente.
 - **deletePdi(Long id)** → Elimina un PDI por ID.

-CentrosController.java

- Administra los centros educativos y culturales.
- Funcionalidades: listar, crear, actualizar y eliminar centros.
- Coordina la relación con PDIs y visitas virtuales.
- Métodos principales:
 - **getAllCentros()** → Devuelve todos los centros registrados.
 - **getCentroById(Integer id)** → Recupera un centro específico.
 - **createCentro(Centros centro)** → Crea un nuevo centro.
 - **updateCentro(Integer id, Centros centro)** → Modifica datos de un centro.
 - **deleteCentro(Integer id)** → Elimina un centro de la base de datos.

-VisitasController.java

- Gestiona tours virtuales asociados a PDIs y centros.
- Relaciona las visitas virtuales con los centros correspondientes.
- Métodos:
 - **getAllVisitas()** → Lista todos los tours virtuales.
 - **getVisitaById(Long id)** → Obtiene un tour específico.

- **createVisita(VisitaVirtual visita)** → Crea un nuevo tour.
- **updateVisita(Long id, VisitaVirtual visita)** → Modifica un tour existente.
- **deleteVisita(Long id)** → Elimina un tour.

-AuthController.java

- Maneja la autenticación y autorización JWT.
- Funciones principales: login, registro de usuarios y generación de tokens.
- Métodos:
 - **authenticateUser(LoginRequest request)** → Valida credenciales y genera JWT.
 - **registerUser(RegisterRequest request)** → Crea un usuario con roles asignados.

-AdminController.java

- Control de usuarios y roles administrativos.
- Permite asignar permisos y roles mediante la interfaz de gestión de permisos.
- Métodos:
 - **asignarRolUsuario(String userId, String rol)** → Asigna un rol a un usuario.
 - **listarUsuarios()** → Devuelve la lista de usuarios y roles actuales.

2.1.2 Service

Objetivo: Encapsular la lógica de negocio y comunicar los controladores con los repositorios.

Archivos y funcionalidades:

-PuntoDeInteresService / PuntoDeInteresServiceImpl.java

- Implementa la lógica de creación, edición, eliminación y búsqueda de PDIs.
- Valida la integridad de los datos y asegura la correcta asociación con centros y usuarios.
- Métodos:
 - **savePdi(PuntoDeInteres pdi)** → Guarda un PDI en la base de datos.

- **findById(Long id)** → Busca un PDI por ID.
- **deletePdi(Long id)** → Borra un PDI.
- **getAllPdis()** → Devuelve lista de PDIs.

-CentroService / CentroServiceImpl.java

- Gestión de centros: creación, actualización y eliminación.
- Implementa validaciones de unicidad de email, teléfono y dirección de cada centro.
- Métodos:
 - **saveCentro(Centros centro)** → Crea o actualiza un centro.
 - **findById(Integer id)** → Busca un centro por ID.
 - **deleteCentro(Integer id)** → Elimina un centro.
 - **getAllCentros()** → Lista todos los centros.

-VisitaVirtualService / VisitaVirtualServiceImpl.java

- Lógica de gestión de visitas virtuales.
- Controla la asociación de visitas a centros y su disponibilidad.
- Métodos:
 - **saveVisita(VisitaVirtual visita)** → Crea o actualiza un tour.
 - **findById(Long id)** → Busca un tour por ID.
 - **deleteVisita(Long id)** → Elimina un tour.
 - **getAllVisitas()** → Lista todos los tours.

-UsuarioService / UsuarioServiceImpl.java

- Gestión de usuarios y roles.
- Métodos de creación, actualización y asignación de permisos.
- Encapsula la lógica de autenticación, cifrado de contraseñas y recuperación de información de usuario.
- Métodos:
 - **saveUsuario(Usuario user)** → Crea un usuario.
 - **findById(Long id)** → Busca usuario por ID.
 - **updateRoles(Long id, Set<RolUsuario> roles)** → Modifica roles de un usuario.
 - **getAllUsuarios()** → Lista usuarios y roles.

2.1.3 Repository

Objetivo: Acceder a la base de datos y realizar operaciones CRUD sobre entidades.

Archivos y funcionalidades:

- **PuntoDeInteresRepository.java:** Operaciones sobre la tabla PDIs.
- **CentroRepository.java:** CRUD de centros.
- **UsuarioRepository.java:** CRUD y búsquedas de usuarios y roles.
- **VisitaVirtualRepository.java:** CRUD de visitas virtuales y consultas por centro.

Estos repositorios utilizan **Spring Data JPA**, permitiendo consultas automáticas a partir de nombres de métodos y anotaciones.

-PuntoDeInteresRepository.java

- Interfaz que extiende `JpaRepository<PuntoDeInteres, Long>`.
- Métodos importantes:
 - **findById(Long id)** → Recupera un PDI por su ID.
 - **findAll()** → Lista todos los PDIs.
 - **save(PuntoDeInteres pdi)** → Guarda o actualiza un PDI.
 - **deleteById(Long id)** → Elimina un PDI por ID.
 - **findByCentro_Id(Integer idCentro)** → Recupera todos los PDIs asociados a un centro específico.

-CentroRepository.java

- Interfaz que extiende `JpaRepository<Centros, Integer>`.
- Métodos importantes:
 - **findById(Integer id)** → Recupera un centro.
 - **findAll()** → Lista todos los centros.
 - **save(Centros centro)** → Crea o actualiza un centro.
 - **deleteById(Integer id)** → Elimina un centro por ID.

- **findByEmailCentro(String email)** → Recupera un centro por su email.

-UsuarioRepository.java

- Interfaz que extiende JpaRepository<Usuario, Long>.
- Métodos importantes:
 - **findById(Long id)** → Recupera un usuario.
 - **findAll()** → Lista todos los usuarios.
 - **save(Usuario user)** → Crea o actualiza un usuario.
 - **deleteById(Long id)** → Elimina un usuario por ID.
 - **findByEmail(String email)** → Recupera usuario por email.

-VisitaVirtualRepository.java

- Interfaz que extiende JpaRepository<VisitaVirtual, Long>.
- Métodos importantes:
 - **findById(Long id)** → Recupera un tour virtual.
 - **findAll()** → Lista todos los tours.
 - **save(VisitaVirtual visita)** → Crea o actualiza un tour.
 - **deleteById(Long id)** → Elimina un tour.
 - **findByCentro_Id(Integer idCentro)** → Recupera tours por centro.

2.1.4 Model

Objetivo: Representar las entidades del sistema y sus relaciones.

Archivos y funcionalidades:

-PuntoDeInteres.java

Representa un PDI con atributos como nombre, creador, centro y contenido JSON.

- Campos: idPdi, centro, creador, nombre, contenidoJson.
- Métodos importantes (Lombok @Data genera getters/setters automáticamente):
 - **getNombre() / setNombre(String)** → Obtiene/modifica nombre del PDI.
 - **getContenidoJson() / setContenidoJson(String)** → Contenido del PDI en JSON.
 - **getCreador() / setCreador(Usuario)** → Usuario creador del PDI.

-Centros.java

Entidad que representa un centro educativo o cultural.

- Campos: idCentros, nombre, emailCentro, telefono, direccion, visitas, puntoDeInteres.
- Métodos importantes:
 - **getNombre() / setNombre(String)** → Nombre del centro.
 - **getVisitas() / setVisitas(Set<VisitaVirtual>)** → Tours virtuales asociados.
 - **getPuntoDeInteres() / setPuntoDeInteres(Set<PuntoDeInteres>)** → PDIs del centro.

-Usuario.java

Modelo de usuario con datos personales, correo y roles asignados.

- Campos: id, nombre, email, password, roles.
- Métodos importantes:
 - **getRoles() / setRoles(Set<RolUsuario>)** → Asigna o consulta roles.
 - **getEmail() / setEmail(String)** → Correo del usuario.

-RolUsuario.java

- Enum con roles: ALUMNO, PROFESOR, ADMIN.
- Métodos generados automáticamente por Java:
 - **valueOf(String)** → Devuelve rol a partir de texto.
 - **values()** → Lista todos los roles.

-VisitaVirtual.java

Representa una visita virtual asociada a un centro.

Incluye URL del contenido virtual, fecha de creación y centro vinculado.

- Campos: id, nombre, url, centro.
- Métodos importantes:
 - **getNombre() / setNombre(String)** → Nombre del tour.
 - **getUrl() / setUrl(String)** → URL del tour virtual.
 - **getCentro() / setCentro(Centros)** → Centro asociado.

2.1.5 Security y Config

Objetivo: Gestionar la autenticación, autorización y protección de endpoints.

Archivos y funcionalidades:

-SecurityBeansConfig.java

Funcionalidad

Define el **PasswordEncoder** que se utilizará en todo el backend para cifrar contraseñas mediante **BCrypt**, uno de los algoritmos más seguros actualmente.

Importancia en el proyecto

- **Es indispensable para el sistema de autenticación.**
- Todo el proceso de registro, login y verificación de contraseñas depende de este bean.
- Spring inyecta este bean automáticamente creando una instancia única.

Métodos importantes

passwordEncoder()

@Bean

```
public PasswordEncoder passwordEncoder() {  
  
    return new BCryptPasswordEncoder();  
  
}
```

Qué hace:

- Crea un bean global de Spring de tipo PasswordEncoder.
- Usa BCrypt (genera hashes fuertes con salt aleatorio).
- Permite cifrar contraseñas de nuevos usuarios.
- Permite verificar contraseñas durante el login.

Relaciones

- Usado en UsuarioServiceImpl para:
 - guardar contraseñas cifradas
 - validar contraseñas del login
- Integrado en SecurityConfig.
- Apoyo directo a JWT (las contraseñas no se desenscriptan, solo se comparan hashes).

-SecurityConfig.java

- Configuración principal de Spring Security.
- Define reglas de acceso según roles, protege endpoints y configura filtros JWT.
- Funciones importantes:
 - Configura autenticación con JWT.
 - Define permisos de acceso por rol (ADMIN, ALUMNO, PROFESOR).
 - Integra JwtAuthenticationFilter.

-JwtAuthenticationFilter.java

- Filtro que intercepta solicitudes HTTP para validar tokens JWT antes de llegar a los controladores.
- Métodos:

- **doFilterInternal(HttpServletRequest, HttpServletResponse, FilterChain)** → Valida token JWT antes de procesar request.

-JwtUtils.java

- Utilidades para crear, validar y extraer información de tokens JWT.
- Métodos:
 - **generateJwtToken(Authentication auth)** → Genera JWT para usuario autenticado.
 - **validateJwtToken(String token)** → Valida token recibido.
 - **getUserNameFromJwtToken(String token)** → Obtiene email/usuario desde JWT.

-AuthEntryPointJwt.java

- Maneja respuestas cuando un usuario no autorizado intenta acceder a un recurso protegido.
- Método:
 - **commence(HttpServletRequest, HttpServletResponse, AuthenticationException)** → Devuelve 401 en caso de acceso no autorizado.

-CorsConfiguration.java

- Configuración de CORS para permitir solicitudes desde el frontend.
- Métodos:
 - **addCorsMappings(CorsRegistry registry)** → Permite solicitudes desde dominios específicos.

-AppRoleConfig.java

- Define roles predeterminados de la aplicación y su asignación inicial.
- Método:
 - **initRoles()** → Crea roles ADMIN, ALUMNO, PROFESOR si no existen.

2.1.6 Visitasvirtuales_backend y Resources

Objetivo: Archivos que definen la configuración de la aplicación y su construcción.

Archivos y funcionalidades:

-application.properties

Contiene configuraciones de conexión a la base de datos, puerto, propiedades de seguridad y parámetros de Spring Boot.

- Configuración principal:
 - **spring.datasource.url** → URL de base de datos.
 - **spring.datasource.username / password** → Credenciales DB.
 - **jwt.secret** → Clave secreta JWT.
 - **server.port** → Puerto de la aplicación.

-pom.xml

Archivo Maven que lista todas las dependencias, plugins y versiones de librerías utilizadas.

Tecnologías principales: Spring Boot, Spring Data JPA, JWT, Lombok, Hibernate, MySQL.

- Dependencias principales:
 - Spring Boot Starter Web, Security, Data JPA, MySQL Driver, Lombok, JWT.
- Plugins: Maven Compiler, Spring Boot Plugin.

-mvnw / mvnw.cmd

Scripts para ejecutar Maven de forma local sin depender de su instalación global.

HELP.md

Archivo de ayuda con instrucciones básicas de configuración y ejecución del backend.

2.1.7 java/com/ies/tour/visitasvirtuales_backend

-VisitasVirtualesBackendApplication.java

Clase principal de arranque del backend.

Inicializa Spring Boot y carga el contexto de la aplicación.

- Método:
 - **main(String[] args)** → Ejecuta SpringApplication.run(...)

2.1.8 Test

-VisitasvirtualesBackendApplicationTests.java

Funcionalidad

Clase de pruebas básica generada automáticamente por Spring Boot.

Importancia

- Garantiza que el contexto de la aplicación se carga sin errores.
- Es el mínimo test necesario para asegurar que el proyecto está bien configurado.

Métodos importantes

contextLoads()

@Test

```
void contextLoads() { }
```

Qué hace:

- Lanza la aplicación en modo test.
- Comprueba que no hay fallos en las configuraciones de Spring Boot.

Relaciones

- Relacionado con el arranque general del backend.
- No interactúa directamente con ninguna otra clase, pero el test fallaría si una clase esencial del proyecto estuviera mal configurada.

2.1.9 Dto

-LoginRequest.java

Funcionalidad

- DTO usado para recibir credenciales del usuario en el login.
- Recibe el *email* y *password* para iniciar sesión.
- Protege la entidad Usuario de exposición directa.
- Asegura que no se pasen datos extra no deseados.

Campos

- email: se usa como username.
- password: contraseña introducida por el usuario.

Importancia

- Es el puente entre la vista y el backend durante el proceso de login.
- Permite desacoplar la petición del modelo Usuario.

-PuntoDeInteresDTO.java

Funcionalidad

- DTO empleado para enviar y recibir información de puntos de interés (PDI), evitando exponer directamente la entidad.
- Permite enviar y recibir información del Punto de Interés sin exponer la entidad completa.
- Incluye validación: evita registros con campos vacíos.
- Gestiona contenido JSON que luego se mapea a la entidad PuntoDeInteres.

Campos

- nombre: nombre del PDI.
- contenidoJson: JSON con multimedia y texto descriptivo.

Validación

- @NotBlank en ambos campos → evita registros inválidos.

Importancia

- Protege la entidad original.
- Garantiza que las peticiones siempre contienen los datos mínimos necesarios.

-RegisterRequest.java

Funcionalidad

- DTO para registrar nuevos usuarios en el sistema.
- Recoge los datos necesarios para registrar un nuevo usuario.

- Permite controlar qué campos son obligatorios/opcionales.
- Se utiliza para crear usuarios sin exponer la contraseña cifrada del modelo.

Campos

- nombre
- email
- password
- rol

Importancia

- Evita exponer directamente la entidad Usuario.
- Permite controlar exactamente qué datos son necesarios en el registro.

2.1.10 Exception

-ResourceNotFoundException.java

Funcionalidad

Excepción personalizada que se lanza cuando un recurso no existe en la base de datos.

Importancia

- Facilita un manejo estandarizado de errores.
- Retorna HTTP **404 NOT FOUND** automáticamente.

2.1.11 Payload/Response

-JwtResponse.java

Funcionalidad

Objeto de respuesta que se envía al frontend luego de un login exitoso.

Incluye:

- token → JWT para autenticación
- email
- rol
- nombre

Importancia

- Es el paquete de información mínima que necesita el frontend para:
 - habilitar vistas según rol
 - saber quién es el usuario
 - guardar el token para futuras peticiones

2.2 Frontend

2.2.1 HTML

Objetivo: Interfaz de usuario para interacción con la aplicación.

Archivos y funcionalidades:

-ViewAdmin.html

- Panel de administración principal para usuarios con rol ADMIN.
- Funcionalidades principales:
 - Menú lateral dinámico que permite navegar entre gestión de PDIs, centros y usuarios.
 - Tabla de usuarios y centros con botones de acción (editar, eliminar, asignar roles).
 - Formularios modales para crear y modificar recursos del sistema.

-crearPdis.html

- Formulario para crear nuevos Puntos de Interés (PDI).
- Funciones destacadas:
 - Input de nombre del PDI y selección de centro asociado.
 - Campo para JSON descriptivo del PDI.
 - Botón "Crear PDI" que invoca funciones de crearPdis.js.
 - Validación de campos antes de enviar al backend.

-centros.html

- Gestión y listado de centros educativos.
- Funcionalidades:
 - Tabla dinámica con todos los centros registrados.
 - Botones para añadir, editar o eliminar centros.
 - Integración con side-menu.js para navegación.

-virtualview.html

- Visualización de las visitas virtuales disponibles.
- Funciones:
 - Renderizado de tours asociados a un centro específico.
 - Posibilidad de seleccionar un tour y visualizar su contenido (URL del tour).
 - Botón de volver a la lista de tours.

-profilesetting.html

- Configuración de perfil de usuario.
- Edición de datos personales y actualización de contraseña.
- Funciones:
 - Modificar información personal: nombre, email y contraseña.
 - Validaciones de campos (email válido, confirmación de contraseña).
 - Botón “Guardar cambios” que invoca profile-setting.js.

-registerForm.html

- Formulario de registro de usuarios.
- Funcionalidades:
 - Campos: nombre, email, contraseña, confirmación de contraseña.
 - Validaciones de inputs antes de enviar al backend.
 - Botón “Registrar” que llama a register-form.js.

-index.html

- Página principal, enlace a vistas de login, registro y navegación del sistema.
- Funciones:
 - Formulario de login con campos de email y contraseña.
 - Botón “Iniciar sesión” que llama a login.js.
 - Mensajes de error dinámicos si credenciales incorrectas.

-ErrorPanel.html

- Panel para mostrar errores en la navegación o procesos del sistema.
- Funcionalidades:
 - Mostrar dinámicamente mensaje de error.
 - Botón de retorno a la página principal.
 - Integración con ErrorPanelScripts.js para gestión de mensajes.

-gestionPermisos.html

- Permite asignar roles a los usuarios.

- Formulario con selección de roles y validación de ID de usuario.
- Funciones:
 - Input para ID de usuario.
 - Select con roles disponibles (Alumno, Profesor).
 - Botón “Asignar rol” que envía la información al backend.
 - Mensajes de éxito/error dinámicos.

2.2.2 JavaScript

Objetivo: Agregar funcionalidad interactiva a la interfaz.

Archivos y funcionalidades:

-crearPdis.js

Maneja la lógica de envío del formulario de creación de PDIs.

- **Función crearPdi()** → Toma los valores del formulario, valida campos y envía request al backend para crear un nuevo PDI.
- **Función mostrarMensaje(msg, tipo)** → Muestra feedback al usuario (éxito/error).
- **Función limpiarFormulario()** → Limpia todos los inputs tras creación exitosa.

-login.js

Controla la autenticación de usuarios y el almacenamiento de tokens.

- **Función iniciarSesion()** → Captura email y contraseña, valida y envía POST al backend para autenticación.
- **Función mostrarError(msg)** → Muestra mensaje de error si credenciales inválidas.
- **Función guardarToken(token)** → Almacena JWT en localStorage tras login exitoso.

-register-form.js

Valida campos y envía la información del registro al backend.

- **Función registrarUsuario()** → Valida inputs del formulario de registro y envía POST al backend.
- **Función mostrarMensaje(msg, tipo)** → Feedback dinámico de éxito o error.
- **Función verificarContraseña()** → Comprueba que la confirmación de contraseña coincide con la principal.

-profile-setting.js

Permite actualizar la información del usuario y aplicar cambios en tiempo real.

- **Función guardarCambiosPerfil()** → Actualiza información personal del usuario en el backend.
- **Función validarInputs()** → Asegura que los campos cumplen reglas (email, longitud de contraseña).
- **Función mostrarMensaje(msg, tipo)** → Feedback al usuario de resultado de la acción.

-side-menu.js

Funcionalidad del menú lateral: apertura, cierre y navegación de vistas.

- **Función toggleMenu()** → Despliega u oculta el menú lateral.
- **Función navegarA(seccion)** → Redirige a la sección seleccionada del panel.
- **Función resaltarActivo()** → Marca el elemento del menú actualmente activo.

-ErrorPanelScripts.js

Lógica para mostrar, ocultar y cerrar mensajes de error dinámicamente.

- **Función mostrarErrorPanel(msg)** → Renderiza mensaje de error en la pantalla.
- **Función ocultarErrorPanel()** → Oculta el panel de error tras un tiempo o acción del usuario.

Notas generales de frontend:

- Todos los formularios incluyen validación de datos y feedback visual.
- Las páginas HTML se apoyan en los scripts JS para interacción dinámica.
- Se utiliza un patrón común de funciones: obtener valores, validar, enviar al backend y mostrar feedback.
- side-menu.js permite la navegación consistente en todas las páginas de administración.

2.2.3 CSS

Objetivo: Estilo y diseño de la interfaz.

Archivos y funcionalidades:

-styles.css

- Define el layout general, tipografía, colores y espaciado.
- Usado en todas las páginas principales.

-ErrorPanelStyles.css

- Estilos específicos para el panel de errores.
- Destaca mensajes y botones de acción en colores de alerta.

2.2.4 Frontend — Imágenes

Objetivo: Recursos visuales usados en la interfaz y visitas virtuales.

Archivos:

- **Davante_logotipo_navy.png**
- **avatar-placeholder.jpg**
- **centro_medac.jpg**
- **dav_blue.png**
- **dav_white.png**

- Carpeta **images/**
 - Contiene todos los archivos .jpg utilizados para representar los centros, puntos de interés y elementos gráficos del sistema.
 - Integradas principalmente en las vistas de **virtualview.html**, **centros.html** y **ViewAdmin.html**.
 - Aporta una experiencia visual para las visitas virtuales y facilita la identificación de los elementos en la interfaz.

2.3 Resumen de jerarquía y diagrama de interacciones

2.3.1 Jerarquía del Frontend

visitasvirtuales_frontend/

└─ errorpanel.html

└─ viewadmin.html

└─ centros.html

└─ crearpdis.html

- ├─ gestionpermisos.html
- ├─ index.html
- ├─ profilesetting.html
- ├─ registerform.html
- ├─ virtualview.html
- ├─ css/
 - ├─ errorpanelstyles.css
 - └─ styles.css
- ├─ images/
 - ├─ Davante_logotipo_navy.png
 - ├─ avatar-placeholder.jpg
 - ├─ centro_medac.jpg
 - ├─ dav_blue.png
 - ├─ dav_white.png
- └─ js/
 - ├─ errorpanelscripts.js
 - ├─ login.js
 - ├─ profile-setting.js
 - ├─ register-form.js
 - ├─ side-menu.js
 - └─ crearpdis.js

- Relación HTML ↔ JS:
 - Cada página HTML tiene asociado uno o varios archivos JS que manejan la interacción de usuario, validaciones de formularios, envío de datos al backend y feedback visual.
- Funciones clave del frontend:
 - Formularios de gestión (crear PDIs, registro, perfil, permisos).
 - Tablas dinámicas de visualización y edición de datos (usuarios, centros, PDIs).
 - Paneles de error y mensajes dinámicos.

- Navegación consistente mediante side-menu.js.

2.3.2 Jerarquía del Backend (src)

visitasvirtuales_backend/

```
├── help.md
├── mvnw / mvnw.cmd
├── pom.xml
├── target/
|   ├── classes/
|       ├── application.properties
|       └── visitasvirtualesbackendapplicationtests.class
└── src/
    ├── visitasvirtualesbackendapplicationtests.java
    └── main/
        ├── resources/
        |   └── application.properties
        └── tour/visitasvirtuales/
            ├── config/
            |   ├── AppRoleConfig.java
            |   ├── CorsConfiguration.java
            |   ├── SecurityBeansConfig.java
            |   └── SecurityConfig.java
```

- └─ controller/
 - | └─ AdminController.java
 - | └─ AuthController.java
 - | └─ CentrosController.java
 - | └─ PuntoDeInteresController.java
 - | └─ Valid.java
 - | └─ VisitasController.java
- └─ dto/
 - | └─ LoginRequest.java
 - | └─ PuntoDeInteresDTO.java
 - | └─ RegisterRequest.java
- └─ exception/
 - | └─ ResourceNotFoundException.java
- └─ model/
 - | └─ Centros.java
 - | └─ PuntoDeInteres.java
 - | └─ RolUsuario.java
 - | └─ Usuario.java
 - | └─ VisitaVirtual.java
- └─ payload/response/
 - | └─ JwtResponse.java
- └─ repository/
 - | └─ CentroRepository.java
 - | └─ PuntoDeInteresRepository.java
 - | └─ UsuarioRepository.java
 - | └─ VisitaVirtualRepository.java

```
├── security/jwt/
|   ├── AuthEntryPointJwt.java
|   ├── JwtAuthenticationFilter.java
|   └── JwtUtils.java
├── service/
|   ├── CentroService.java
|   ├── CentroServiceImpl.java
|   ├── PuntoDeInteresService.java
|   ├── PuntoDeInteresServiceImpl.java
|   ├── UsuarioService.java
|   ├── UsuarioServiceImpl.java
|   ├── VisitaVirtualService.java
|   └── VisitaVirtualServiceImpl.java
└── VisitasVirtualesBackendApplication.java
```

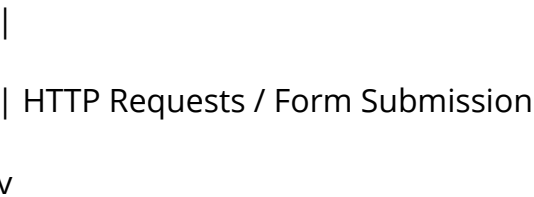
- **Relaciones principales:**

Frontend HTML/JS <---- REST API ----> Backend Controllers ----> Services ----> Repositories ----> Database

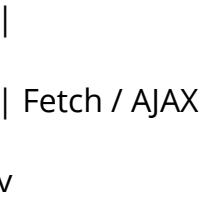
1. **Controladores (Controllers):** reciben solicitudes HTTP del frontend y llaman a los servicios.
2. **Servicios (Services):** lógica de negocio, validaciones y coordinación de repositorios.
3. **Repositorios (Repositories):** acceden a la base de datos, ejecutan consultas CRUD.
4. **Modelos (Models):** representan entidades de la base de datos (Usuario, Centro, PuntoDeInteres, etc.).
5. **Seguridad (Security):** autenticación JWT, configuración CORS, roles de usuario.
6. **Configuración y arranque:** VisitasVirtualesBackendApplication.java inicializa el backend y Spring Boot.

2.3.3 Diagrama de Interacciones Simplificado

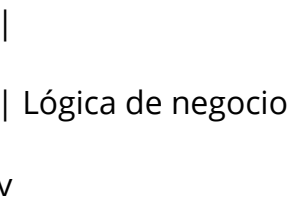
[Usuario / Navegador]



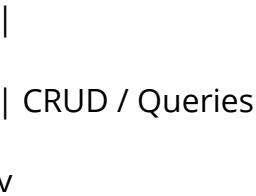
[Frontend HTML + JS]



[Backend Controllers]



[Backend Services]



[Repositories] ---> [Base de datos (Plastic SCM)]



[Models]

- Las páginas HTML y los scripts JS interactúan directamente con los Controllers mediante fetch API o formularios POST/GET.
- Los Services encapsulan reglas de negocio y validaciones.
- Los Repositories realizan operaciones en la base de datos.
- La seguridad (JWT + roles) asegura que solo usuarios autorizados puedan acceder a ciertas rutas.
- Las entidades (Model) representan la estructura de la base de datos y se mapean automáticamente mediante JPA/Hibernate.

2.4 Arquitectura del Sistema

- El backend del proyecto sigue una arquitectura **en capas**, basada en buenas prácticas de **Spring Boot** y organizada por módulos según su responsabilidad.
- Cada bloque cumple una función específica para garantizar un sistema mantenible, escalable y seguro.

1. Resources

Descripción

Contiene los recursos estáticos y de configuración del proyecto:

- **application.properties / application.yml**
Configura la conexión a la base de datos, puertos, propiedades de JPA/Hibernate, configuración del JWT, etc.
- Archivos estáticos o plantillas (si existieran).
- Scripts SQL si el proyecto los incluye.

Importancia

Es donde se define el comportamiento del backend a nivel de infraestructura: base de datos, variables de entorno, políticas de seguridad, etc.

2. Config

Descripción

Contiene las clases que configuran servicios fundamentales del sistema.

Archivos relevantes

- **SecurityBeansConfig.java**
Registra el bean PasswordEncoder (BCrypt), necesario para encriptar contraseñas.

Rol en la arquitectura

- Proporciona componentes globales reutilizables.
- Decouple configuraciones del core de la lógica del sistema.

Dependencias

- Usado por el módulo **security/jwt** para validar credenciales.
- Utilizado por el módulo **service** para registrar o autenticar usuarios.

3. Controller

Descripción

Los controladores exponen la API REST hacia el exterior.

Funciones

- Reciben peticiones HTTP.
- Validan datos de entrada.
- Llaman a los servicios correspondientes.
- Devuelven respuestas encapsuladas (DTOs, payloads).

Archivo relevante

- **Valid.java**

Funciona como marcador dentro del proyecto.

Importancia

Es la capa más cercana al cliente (frontend o terceros), define endpoints y controla el flujo de entrada/salida.

4. DTO

Descripción

Son **objetos de transferencia de datos** que se utilizan para enviar o recibir información entre el cliente y el backend.

Archivos relevantes

- **LoginRequest.java** → recibe email y password.
- **RegisterRequest.java** → recibe nombre, email, password y rol.
- **PuntoDeInteresDTO.java** → transfiere datos del Punto de Interés.

Responsabilidad

- Evitar exponer las entidades del modelo directamente.
- Controlar qué campos llegan desde el frontend.
- Ordenar la información que se devuelve.

Importancia

Previene filtraciones de datos y desacopla el modelo de la estructura REST.

5. Exception

Descripción

Contiene las clases de manejo de errores personalizados.

Archivo relevante

- **ResourceNotFoundException.java**
Lanzada cuando un elemento buscado en la BD no existe.

Responsabilidad

- Evitar errores genéricos.
- Permitir al controlador devolver mensajes claros y estructurados.
- Mantener un flujo fiable ante fallos.

6. Model

Descripción

Contiene las entidades JPA que representan objetos reales del dominio.

Archivos relevantes

- **Usuario**
- **Centros**
- **PuntoDeInteres**
- **VisitaVirtual**
- Otros modelos almacenados en las tablas del proyecto.

Responsabilidad

- Representación exacta de las tablas de la base de datos.
- Gestionadas por Hibernate (ORM) para consultas y persistencia.

Importancia

Es el corazón del backend: define Estructura → Relaciones → Integridad.

7. Payload/Response

Descripción

Clases usadas para **responder al cliente**, especialmente en procesos de autenticación.

Archivo relevante

- **JwtResponse.java**
Incluye token JWT, email, rol, etc.

Responsabilidad

- Encapsular datos enviados como respuesta.
- Permitir respuestas limpias y estándar.

Relación

Usado por:

- Controladores de autenticación.
- Servicios que generan tokens.

8. Repository

Descripción

Capa de acceso a datos mediante **Spring Data JPA**.

Responsabilidad

- Proporcionar métodos CRUD hacia la base de datos.
- Custom queries si se necesitan.

Relación con otros módulos

- Llamados por los **services**, nunca por controladores.
- Reciben entidades del módulo **model**.

Importancia

Define cómo se interactúa con la BD sin escribir SQL explícito en la mayoría de casos.

9. Security/Jwt

Descripción

Bloque encargado de gestionar la seguridad mediante tokens JWT.

Incluye:

- Filtros JWT
- Proveedores de autenticación
- Token utilities
- Configuración de seguridad HTTP

Responsabilidad

- Validar credenciales.
- Generar tokens JWT al iniciar sesión.
- Proteger endpoints.
- Comprobar roles o permisos.

Relación

- Usa PasswordEncoder del bloque **config**.
- Interactúa con el repositorio de usuarios para autenticar.
- Devuelve JwtResponse al login.
- Controladores dependen de sus reglas de seguridad.

10. Service

Descripción

Implementa la **lógica de negocio** del proyecto.

Funciones

- Gestión de usuarios (registro, login, validación).
- Gestión de centros.
- Gestión de visitas virtuales.
- Gestión de puntos de interés.
- Conversión de entidades ↔ DTO.

Responsabilidad

- Contiene reglas del sistema.
- Maneja transacciones.
- Interactúa con repositorios.
- Devuelve información lista para los controladores.

Importancia

Es la capa decisiva: controla cómo funciona el sistema internamente.

Relación entre bloques (Resumen visual)

CLIENTE (Frontend / API client)

|

v

controller

|

| usa DTO para entrada y Payload/Response para salida

v

service <----- security/jwt (valida accesos)

|

v

repository

|

v

model <----> Base de datos

Los módulos **exception**, **config** y **resources** trabajan en segundo plano proporcionando estructura, errores y configuración.

2.5 Diagrama de Componentes (Resumen)

- **Frontend:** Navegador → HTML/JS → APIs REST.
- **Backend:** Spring Boot
 - Controllers → Services → Repositories → Base de datos Plastic SCM.
- **Seguridad:** JWT + Spring Security.
- **Base de datos:** Gestionada con Plastic SCM (control de versiones del proyecto) y scripts SQL para migraciones.

2.6 Estructura del Repositorio y Convenciones de Código

- Backend en visitasvirtuales_backend/
 - Estructura modular por capas.
 - Uso de **nomenclatura CamelCase** para clases y métodos descriptivos.
 - Separación de DTOs (dto/) y Responses (payload/response/).
- Frontend en visitasvirtuales_frontend/
 - HTML por vista (index.html, ViewAdmin.html, etc.)
 - JS por funcionalidad (login.js, crearPdis.js, profile-setting.js)
 - CSS por estilo general o específico de vista (styles.css, ErrorPanelStyles.css)

2.7 Configuración del Entorno de Desarrollo

- **Backend:**
 - Java 17, Spring Boot 3.x, Maven.
 - Variables de entorno configuradas en application.properties.
 - Dependencias gestionadas en pom.xml.
 - Proyecto compilado con mvn clean install.
- **Frontend:**
 - Navegador moderno con soporte HTML5, CSS3 y JS.
 - Integración directa con backend mediante endpoints REST.
- **Base de datos:**
 - Gestionada con Plastic SCM y scripts de migración.
- **DevOps / Despliegue:**
 - Puede ejecutarse en servidor local o remoto.
 - Uso de Unity como launcher del proyecto.

2.8 Servicios (Services)

Los servicios implementan la lógica de negocio y actúan como intermediarios entre los controladores y los repositorios.

2.8.1 PuntoDeInteresService / PuntoDeInteresServiceImpl

- **Funcionalidad general:**

Gestiona los Puntos de Interés (PDIs) de cada centro.
- **Métodos principales:**
 - **crearPDI(PuntoDeInteres pdi):** Guarda un nuevo PDI en la base de datos.
 - **actualizarPDI(Long id, PuntoDeInteres pdi):** Actualiza los datos de un PDI existente.
 - **eliminarPDI(Long id):** Elimina un PDI por su ID.
 - **listarPDIsPorCentro(Integer idCentro):** Devuelve todos los PDIs de un centro.
- **Relaciones:**

- Relación ManyToOne con **Centros** y **Usuario** creador.

2.8.2 UsuarioService / UsuarioServiceImpl

- **Funcionalidad general:**

Gestiona usuarios y roles dentro del sistema.

- **Métodos principales:**

- **registrarUsuario(RegisterRequest request):** Registra un nuevo usuario con roles.
- **buscarUsuarioPorId(Long id):** Recupera un usuario.
- **asignarRol(Long idUsuario, String rol):** Asigna un rol a un usuario.

- **Notas:**

- Gestiona validaciones de datos y duplicados.

2.8.3 CentroService / CentroServiceImpl

- **Funcionalidad general:**

CRUD de centros educativos.

- **Métodos principales:**

- **crearCentro(Centros centro), actualizarCentro, eliminarCentro.**
- **listarCentros():** Devuelve todos los centros.

- **Relaciones:**

- Un centro tiene varios PDIs y varias visitas virtuales.

2.8.4 VisitaVirtualService / VisitaVirtualServiceImpl

- **Funcionalidad general:**

Gestiona visitas virtuales y la asociación con PDIs.

- **Métodos principales:**

- **crearVisita(VisitaVirtual visita), actualizarVisita, eliminarVisita.**
- **listarVisitasPorCentro(Integer idCentro):** Recupera visitas asociadas a un centro.

- **Notas:**

- Permite filtrar PDIs según criterios de usuario o centro.

2.9 Repositorio (Repository)

Los repositorios son responsables **del acceso a datos**, utilizando Spring Data JPA. Implementan operaciones CRUD y consultas personalizadas.

2.9.1 PuntoDeInteresRepository

- **Funcionalidad:**

Operaciones sobre PDIs.

- **Consultas destacadas:**
 - **findByCentro(Centros centro):** Devuelve todos los PDIs de un centro.

2.9.2 UsuarioRepository

- **Funcionalidad:**
CRUD de usuarios.
- **Consultas destacadas:**
 - **findByEmail(String email):** Buscar usuario por correo.
 - **existsByUsername(String username):** Validación de duplicados.

2.9.3 CentroRepository

- **Funcionalidad:**
CRUD de centros.
- **Consultas destacadas:**
 - **findByNombre(String nombre):** Buscar centro por nombre.

2.9.4 VisitaVirtualRepository

- **Funcionalidad:**
CRUD de visitas virtuales.
- **Consultas destacadas:**
 - **findByCentro(Centros centro):** Filtra visitas por centro.

2.10 Modelos (Entities)

Los modelos representan las **entidades de la base de datos** y son gestionados mediante JPA/Hibernate.

2.10.1 PuntoDeInteres.java

- **Funcionalidad:**
Representa un Punto de Interés de un centro.
- **Atributos principales:**
 - **idPdi:** Identificador único.
 - **centro:** Asociación ManyToOne con **Centros**.
 - **creador:** Usuario que creó el PDI.
 - **nombre:** Nombre del PDI.
 - **contenidoJson:** Información adicional en formato JSON.
- **Notas:**
 - Permite almacenar descripciones extensas en **contenidoJson**.
 - La relación con **Centros** y **Usuario** facilita filtros y permisos.

2.10.2 Centros.java

- **Funcionalidad:**
Representa un centro educativo.
- **Atributos principales:**
 - **idCentros, nombre, emailCentro, telefono, direccion.**
 - Relaciones OneToMany con PDIs y Visitas Virtuales.
- **Notas:**
 - **fetch = EAGER** en PDIs asegura que siempre se cargan junto al centro.
 - Orphan removal en visitas virtuales para mantener consistencia.

2.10.3 Usuario.java

- **Funcionalidad:**
Representa a los usuarios del sistema.
- **Atributos principales:**
 - **idUsuario, username, email, password.**
 - Roles mediante **RolUsuario** y relaciones.
- **Métodos relevantes:**
 - Validación de roles y credenciales.

2.10.4 RolUsuario.java

- **Funcionalidad:**
Define los roles de usuario (**ALUMNO, PROFESOR, ADMIN**).
- **Relaciones:**
 - Muchos a uno con **Usuario**.

2.10.5 VisitaVirtual.java

- **Funcionalidad:**
Representa visitas virtuales asociadas a centros y PDIs.
- **Atributos importantes:**
 - **id, centro, nombre, descripcion, pdis.**
- **Notas:**
 - Permite filtrar o asignar PDIs a cada visita virtual.

2.11 Seguridad (Security)

El módulo de seguridad protege el backend usando **JWT** y **Spring Security**.

2.11.1 SecurityConfig.java

- Configura filtros de seguridad, endpoints públicos y privados.
- Define autenticación basada en JWT y roles.

2.11.2 JwtAuthenticationFilter.java

- Intercepta solicitudes HTTP.
- Valida tokens JWT y asigna contexto de seguridad a Spring.

2.11.3 JwtUtils.java

- Genera, valida y analiza tokens JWT.
- Métodos principales: **generateJwtToken()**, **validateJwtToken()**, **getUserNameFromJwtToken()**.

2.11.4 AuthEntryPointJwt.java

- Controla errores de autenticación y accesos no autorizados.
- Devuelve mensajes HTTP 401 cuando corresponde.

2.11.5 CorsConfiguration.java

- Configura políticas CORS para permitir comunicación entre frontend y backend.
- Permite definir dominios permitidos, métodos y cabeceras.

2.11.6 AppRoleConfig.java

- Enumera los roles disponibles.
- Facilita la asignación de roles y validaciones en servicios.

2.12 DTO

Los DTO son **objetos de transferencia de datos** que se utilizan para **intercambiar información entre el frontend y el backend** sin exponer directamente las entidades del modelo.

2.12.1 LoginRequest.java

Métodos

Generados automáticamente por Lombok:

- getEmail()
- setEmail(String)
- getPassword()

- setPassword(String)

Relaciones

- Consumido por el controlador de autenticación (AuthController).
- Usado por UsuarioService para validar credenciales.
- Conecta con la generación de JWT.

2.12.2 PuntoDeInteresDTO.java

Métodos

Generados por Lombok:

- getters/setters
- toString()
- equals() y hashCode()

Relaciones

- Usado en PuntoDeInteresController.
- Convertido a entidad en PuntoDeInteresServiceImpl.

2.12.3 RegisterRequest.java

Métodos

Generados automáticamente por Lombok.

Relaciones

- Consumido por el endpoint de registro.
- Usado por UsuarioServiceImpl en la creación de usuarios.
- Interactúa con PasswordEncoder y gestión de roles.

2.13 Exception

El bloque exception contiene clases que representan errores personalizados del proyecto.

Función principal en el proyecto

- Estandariza cómo se manejan los errores.
- Permite lanzar mensajes claros cuando algo falla.
- Facilita una respuesta HTTP coherente (por ejemplo, 404 para elementos no encontrados).
- Evita usar mensajes genéricos o stacktraces inseguros.

2.13.1 ResourceNotFoundException.java

Métodos

Constructor ResourceNotFoundException(String message)

```
public ResourceNotFoundException(String message) {  
    super(message);  
}
```

}

Qué hace:

- Envía el mensaje personalizado a la capa superior.
- Permite mensajes como:
 - "Usuario no encontrado"
 - "Punto de interés no existe"

Relaciones

- Usada en todos los servicios:
 - UsuarioServiceImpl
 - PuntoDeInteresServiceImpl
 - VisitaVirtualServiceImpl
- Manejada por Spring → responde en JSON.

2.14 Payload/Response

Este bloque contiene los objetos que el backend envía al frontend como **respuesta estructurada**.

Son parecidos a los DTO, pero usados **de salida**, no de entrada.

Función principal en el proyecto

- Define un formato específico para las respuestas HTTP.
- Evita exponer directamente las entidades internas.
- Facilita que el frontend reciba solo lo que necesita.

2.14.1 JwtResponse.java

Métodos importantes

Métodos personalizados (no generados por Lombok):

- getToken() / setToken()
- getEmail() / setEmail()
- getRol() / setRol()
- getNombre() / setNombre()

Relaciones

- Producido por el controlador de login.
- Usado por el frontend para almacenar datos en sessionStorage o localStorage.
- Ligado a JwtUtils (generación del token).

2.15 Buenas prácticas y convenciones

- Uso de **interfaces para servicios (Service)** y su implementación (**ServiceImpl**) para asegurar **inversión de dependencias**.
- Nomenclatura descriptiva para métodos y variables.
- Separación de responsabilidades:
 - Controllers → entrada/salida de datos.
 - Services → lógica de negocio.
 - Repositories → acceso a datos.
- Uso de DTOs (**registerRequest**, **loginRequest**, **PuntoDeInteresDTO**) para evitar exponer entidades directamente al frontend.

2.16 Relaciones y jerarquía de clases

- **Usuario** → tiene 1..N **RolUsuario**
- **Centros** → tiene 1..N **PuntoDelInteres** y 1..N **VisitaVirtual**
- **PuntoDelInteres** → pertenece a 1 **Centro** y 1 **Usuario** creador
- **VisitaVirtual** → pertenece a 1 **Centro** y contiene varios PDIs
- La seguridad se aplica sobre todas las operaciones críticas, usando roles y JWT.

- **Estructura del repositorio GitHub**

visitasvirtuales/

```
├─ visitasvirtuales_backend/
|   └─ src/
|       └─ main/
|           └─ java/com/ies/tour/visitasvirtuales_backend/
|               └─ config/
|                   └─ AppRoleConfig.java
|                   └─ CorsConfiguration.java
|                   └─ SecurityBeansConfig.java
|                   └─ SecurityConfig.java
|                   └─ controller/
|                       └─ AdminController.java
|                       └─ AuthController.java
|                       └─ CentrosController.java
```

- | | | | | | | — PuntoDeInteresController.java
- | | | | | | | — Valid.java
- | | | | | | | — VisitasController.java
- | | | | | | — dto/
- | | | | | | | — LoginRequest.java
- | | | | | | | — PuntoDeInteresDTO.java
- | | | | | | | — RegisterRequest.java
- | | | | | | — exception/
- | | | | | | | — ResourceNotFoundException.java
- | | | | | | — model/
- | | | | | | | — Centros.java
- | | | | | | | — PuntoDeInteres.java
- | | | | | | | — RolUsuario.java
- | | | | | | | — Usuario.java
- | | | | | | | — VisitaVirtual.java
- | | | | | | — payload/response/
- | | | | | | | — JwtResponse.java
- | | | | | | — repository/
- | | | | | | | — CentroRepository.java
- | | | | | | | — PuntoDeInteresRepository.java
- | | | | | | | — UsuarioRepository.java
- | | | | | | | — VisitaVirtualRepository.java
- | | | | | | — security/jwt/
- | | | | | | | — AuthEntryPointJwt.java
- | | | | | | | — JwtAuthenticationFilter.java
- | | | | | | | — JwtUtils.java

```
| | | | | └─ service/
| | | | | | └─ CentroService.java
| | | | | | └─ CentroServiceImpl.java
| | | | | | └─ PuntoDeInteresService.java
| | | | | | └─ PuntoDeInteresServiceImpl.java
| | | | | | └─ UsuarioService.java
| | | | | | └─ UsuarioServiceImpl.java
| | | | | | └─ Usuarioservice.java
| | | | | | └─ VisitaVirtualService.java
| | | | | | └─ VisitaVirtualServiceImpl.java
| | | | └─ VisitasvirtualesBackendApplication.java
| | └─ resources/
| | | └─ application.properties
| └─ test/java/com/ies/tour/visitasvirtuales_backend/
| | └─ VisitasvirtualesBackendApplicationTests.java
| └─ target/
| | └─ classes/
| | | └─ com/ies/tour/visitasvirtuales_backend/
| | | | └─ config/
| | | | | └─ AppRoleConfig.class
| | | | | └─ CorsConfiguration.class
| | | | | └─ SecurityBeansConfig.class
| | | | | └─ SecurityConfig.class
| | | | └─ controller/
| | | | | └─ AdminController.class
| | | | | └─ AuthController.class
```

- | | | | | | | — CentrosController.class
- | | | | | | | — PuntoDeInteresController.class
- | | | | | | | — Valid.class
- | | | | | | | — VisitasController.class
- | | | | | | — dto/
 - | | | | | | | — LoginRequest.class
 - | | | | | | | — PuntoDeInteresDTO.class
 - | | | | | | | — RegisterRequest.class
- | | | | | | — exception/
 - | | | | | | | — ResourceNotFoundException.class
- | | | | | | — model/
 - | | | | | | | — Centros.class
 - | | | | | | | — PuntoDeInteres.class
 - | | | | | | | — RolUsuario.class
 - | | | | | | | — Usuario.class
 - | | | | | | | — VisitaVirtual.class
- | | | | | | — payload/response/
 - | | | | | | | — JwtResponse.class
- | | | | | | — repository/
 - | | | | | | | — CentroRepository.class
 - | | | | | | | — PuntoDeInteresRepository.class
 - | | | | | | | — UsuarioRepository.class
 - | | | | | | | — VisitaVirtualRepository.class
- | | | | | | — security/jwt/
 - | | | | | | | — AuthEntryPointJwt.class
 - | | | | | | | — JwtAuthenticationFilter.class

- | | | | | | └─ JwtUtils.class
- | | | | | └─ service/
- | | | | | | └─ CentroService.class
- | | | | | | └─ CentroServiceImpl.class
- | | | | | | └─ PuntoDeInteresService.class
- | | | | | | └─ PuntoDeInteresServiceImpl.class
- | | | | | | └─ UsuarioService.class
- | | | | | | └─ UsuarioServiceImpl.class
- | | | | | | └─ Usuarioservice.class
- | | | | | | └─ VisitaVirtualService.class
- | | | | | | └─ VisitaVirtualServiceImpl.class
- | | | | | └─ VisitasvirtualesBackendApplication.class
- | | └─ application.properties
- | | └─ test-classes/com/ies/tour/visitasvirtuales_backend/
- | | | └─ VisitasvirtualesBackendApplicationTests.class
- | └─ HELP.md
- | └─ mvnw
- | └─ mvnw.cmd
- | └─ pom.xml
- |
- └─ visitasvirtuales_frontend/
- | └─ css/
- | | └─ ErrorPanelStyles.css
- | | └─ styles.css
- | └─ images/
- | | └─ Davante_logotipo_navy.png

- | | | └─ avatar-placeholder.jpg
- | | | └─ centro_medac.jpg
- | | | └─ dav_blue.png
- | | | └─ dav_white.png
- | | └─ js/
- | | | └─ ErrorPanelScripts.js
- | | | └─ crearPdis.js
- | | | └─ login.js
- | | | └─ profile-setting.js
- | | | └─ register-form.js
- | | | └─ side-menu.js
- | | └─ ErrorPanel.html
- | | └─ ViewAdmin.html
- | | └─ centros.html
- | | └─ crearPdis.html
- | | └─ gestionPermisos.html
- | | └─ index.html
- | | └─ profilesetting.html
- | | └─ registerForm.html
- | | └─ virtualview.html
- |
- └─ README.md

- **Jerarquía y estructura del backend**

Objetivo: Comprender cómo están organizados los archivos y carpetas para un fácil mantenimiento.

```

└─ visitasvirtuales_backend/
    │   └─ src/
    │       │   └─ main/
    │       │       │   └─ java/com/ies/tour/visitasvirtuales_backend/
    │       │       │       │   └─ config/
    │       │       │       │       │   └─ AppRoleConfig.java
    │       │       │       │       │   └─ CorsConfiguration.java
    │       │       │       │       │   └─ SecurityBeansConfig.java
    │       │       │       │       │   └─ SecurityConfig.java
    │       │       │       │       └─ controller/
    │       │       │       │           │   └─ AdminController.java
    │       │       │       │           │   └─ AuthController.java
    │       │       │       │           │   └─ CentrosController.java
    │       │       │       │           │   └─ PuntoDeInteresController.java
    │       │       │       │           │   └─ Valid.java
    │       │       │       │           │   └─ VisitasController.java
    │       │       │       │       └─ dto/
    │       │       │       │           │   └─ LoginRequest.java
    │       │       │       │           │   └─ PuntoDeInteresDTO.java
    │       │       │       │           │   └─ RegisterRequest.java
    │       │       │       │       └─ exception/
    │       │       │       │           └─ ResourceNotFoundException.java

```

```
| | | | | └─ model/
| | | | | | └─ Centros.java
| | | | | | └─ PuntoDeInteres.java
| | | | | | └─ RolUsuario.java
| | | | | | └─ Usuario.java
| | | | | | └─ VisitaVirtual.java
| | | | | └─ payload/response/
| | | | | | └─ JwtResponse.java
| | | | | └─ repository/
| | | | | | └─ CentroRepository.java
| | | | | | └─ PuntoDeInteresRepository.java
| | | | | | └─ UsuarioRepository.java
| | | | | | └─ VisitaVirtualRepository.java
| | | | | └─ security/jwt/
| | | | | | └─ AuthEntryPointJwt.java
| | | | | | └─ JwtAuthenticationFilter.java
| | | | | | └─ JwtUtils.java
| | | | | └─ service/
| | | | | | └─ CentroService.java
| | | | | | └─ CentroServiceImpl.java
| | | | | | └─ PuntoDeInteresService.java
| | | | | | └─ PuntoDeInteresServiceImpl.java
| | | | | | └─ UsuarioService.java
| | | | | | └─ UsuarioServiceImpl.java
| | | | | | └─ Usuarioservice.java
| | | | | | └─ VisitaVirtualService.java
```

```
| | | | | └─ VisitaVirtualServiceImpl.java
| | | | | └─ VisitasvirtualesBackendApplication.java
| | | └─ resources/
| | |   └─ application.properties
| └─ test/java/com/ies/tour/visitasvirtuales_backend/
| | └─ VisitasvirtualesBackendApplicationTests.java
| └─ target/
| | └─ classes/
| | | └─ com/ies/tour/visitasvirtuales_backend/
| | | | └─ config/
| | | | | └─ AppRoleConfig.class
| | | | | └─ CorsConfiguration.class
| | | | | └─ SecurityBeansConfig.class
| | | | | └─ SecurityConfig.class
| | | | └─ controller/
| | | | | └─ AdminController.class
| | | | | └─ AuthController.class
| | | | | └─ CentrosController.class
| | | | | └─ PuntoDeInteresController.class
| | | | | └─ Valid.class
| | | | | └─ VisitasController.class
| | | | └─ dto/
| | | | | └─ LoginRequest.class
| | | | | └─ PuntoDeInteresDTO.class
| | | | | └─ RegisterRequest.class
| | | | └─ exception/
```

| | | | | └─ ResourceNotFoundException.class

| | | | | └─ model/

| | | | | | └─ Centros.class

| | | | | | └─ PuntoDeInteres.class

| | | | | | └─ RolUsuario.class

| | | | | | └─ Usuario.class

| | | | | | └─ VisitaVirtual.class

| | | | | └─ payload/response/

| | | | | | └─ JwtResponse.class

| | | | | └─ repository/

| | | | | | └─ CentroRepository.class

| | | | | | └─ PuntoDeInteresRepository.class

| | | | | | └─ UsuarioRepository.class

| | | | | | └─ VisitaVirtualRepository.class

| | | | | └─ security/jwt/

| | | | | | └─ AuthEntryPointJwt.class

| | | | | | └─ JwtAuthenticationFilter.class

| | | | | | └─ JwtUtils.class

| | | | | └─ service/

| | | | | | └─ CentroService.class

| | | | | | └─ CentroServiceImpl.class

| | | | | | └─ PuntoDeInteresService.class

| | | | | | └─ PuntoDeInteresServiceImpl.class

| | | | | | └─ UsuarioService.class

| | | | | | └─ UsuarioServiceImpl.class

| | | | | | └─ Usuarioservice.class

```
| | | | | | └─ VisitaVirtualService.class
| | | | | | └─ VisitaVirtualServiceImpl.class
| | | | | └─ VisitasvirtualesBackendApplication.class
| | └─ application.properties
| | └─ test-classes/com/ies/tour/visitasvirtuales_backend/
| | | └─ VisitasvirtualesBackendApplicationTests.class
| └─ HELP.md
| └─ mvnw
| └─ mvnw.cmd
| └─ pom.xml
```

2.17 Frontend

El frontend está desarrollado en **HTML5, CSS3 y JavaScript**. Se aloja en la carpeta **visitasvirtuales_frontend** y se comunica con el backend mediante **API REST**.

2.17.1 Estructura de carpetas

visitasvirtuales_frontend/

```
└─ errorpanel.html
```

```
└─ viewadmin.html
```

```
└─ centros.html
```

- └─ crearpdis.html
- └─ gestionpermisos.html
- └─ index.html
- └─ profilessetting.html
- └─ registerform.html
- └─ virtualview.html
- └─ css/
 - └─ errorpanelstyles.css
 - └─ styles.css
- └─ images/
 - └─ Davante_logotipo_navy.png
 - └─ avatar-placeholder.jpg
 - └─ centro_medac.jpg
 - └─ dav_blue.png
 - └─ dav_white.png
- └─ js/
 - └─ errorpanelscripts.js
 - └─ login.js
 - └─ profile-setting.js
 - └─ register-form.js
 - └─ side-menu.js
 - └─ crearpdis.js

2.18 Descripción de los archivos y funcionalidades

2.18.1 index.html

- Página de inicio.
- Permite **login** de usuarios.
- Redirige a **viewadmin.html** o **virtualview.html** según rol.

2.18.2 viewadmin.html

- Panel administrativo.
- Permite gestionar usuarios, PDIs y visitas virtuales.
- Contiene enlaces a **crearPdis.html** y **gestionpermisos.html**.

2.18.3 centros.html

- Muestra la lista de centros educativos.
- Permite seleccionar un centro para ver sus PDIs y visitas virtuales.

2.18.4 crearpdis.html

- Formulario para crear un **Punto de Interés**.
- Incluye campos: nombre, descripción y JSON adicional.
- Se comunica con **PuntoDeInteresController** vía API POST.

2.18.5 gestionpermisos.html

- Formulario para asignar **roles a usuarios**.
- Permite seleccionar el rol (Alumno, Profesor) y el ID de usuario.
- Se comunica con **AdminController** para guardar cambios.

2.18.6 profilesetting.html

- Permite editar la información de perfil del usuario.
- Cambios sincronizados con **UsuarioService**.

2.18.7 registerform.html

- Registro de nuevos usuarios.
- Campos: username, email, password y confirmación.
- Se valida en frontend y backend antes de almacenar.

2.18.8 virtualview.html

- Visualización de **visitas virtuales** para alumnos y profesores.
- Muestra los PDIs asociados a cada visita virtual.

2.18.9 errorpanel.html

- Página de error genérica.

- Se utiliza para mostrar mensajes de error en la aplicación.

2.19 JavaScript

Los scripts gestionan la **interactividad, validaciones y comunicación con el backend**.

- **login.js**: valida formulario de login y realiza llamada API para autenticación.
- **register-form.js**: valida registro y envía datos al backend.
- **profile-setting.js**: actualiza perfil del usuario.
- **crearPdis.js**: envía datos de un nuevo PDI al backend.
- **side-menu.js**: gestiona la navegación lateral.
- **ErrorPanelScripts.js**: maneja errores y alertas en la UI.

2.20 CSS

- **styles.css**: estilos globales del frontend (tipografía, colores, layout).
- **ErrorPanelStyles.css**: estilos específicos para la gestión de errores.

2.21 Jerarquía y flujo

1. Usuario inicia sesión en **index.html**.
2. Según rol:
 - Administrador → **viewadmin.html** → gestión de PDIs, usuarios y permisos.
 - Alumno/Profesor → **virtualview.html** → acceso a visitas virtuales.
3. Formularios (**crearpdis.html**, **registerform.html**, **profilesetting.html**) interactúan con backend via JS.
4. Errores se muestran en **errorpanel.html**.

2.22 Relación entre Backend y Frontend

Comunicación

El frontend interactúa con el backend exclusivamente mediante:

- Fetch API (JavaScript)
- JSON
- JWT (Bearer Token)
- Rutas REST expuestas por los controladores:

Por ejemplo:

Autenticación:

POST /api/auth/login

Centros:

GET /api/centros/all

POST /api/centros/save

Puntos de Interés:

POST /api/pdis/create

GET /api/pdis/centro/{id}

Visitas Virtuales:

GET /api/visitas/centro/{id}

Seguridad

1. Interceptado por JwtAuthenticationFilter
2. Validación con JwtUtils
3. Bloqueo/Permisos con SecurityConfig
4. Manejo de errores con AuthEntryPointJwt

Persistencia

El backend usa:

- Repositorios JPA (*.Repository.class)
- Entidades JPA (model/*.class)
- Servicios (service/*.class)

Hibernate se encarga de:

- abrir transacciones
- mapear entidades
- generar consultas
- sincronizar con la base de datos

2.23 Archivos .class

Los archivos con extensión **.class** representan **el bytecode Java compilado** a partir del código fuente .java. Son generados automáticamente por Maven (o por el compilador de Java) durante el proceso de construcción del proyecto y nunca deben modificarse manualmente.

Estos archivos se encuentran dentro de:

visitasvirtuales_backend/target/classes/com/ies/tour/visitasvirtuales_backend

2.23.1 Función de los Archivos .class

Los .class son **la forma ejecutable del código Java**.

El flujo es:

1. Archivos .java
2. Maven ejecuta javac
3. Se generan archivos .class
4. La JVM ejecuta esos .class para iniciar la aplicación

Por tanto, **los .class son imprescindibles para que la aplicación se ejecute, pero no se editan**.

2.23.2 Organización de los .class en el proyecto

La estructura de los .class replica al 100% la estructura de paquetes .java.

Ejemplo:

Código Fuente .java	Bytecode .class
src/main/java/.../controller/AdminController.java	target/classes/.../controller/AdminController.class

Esto facilita la carga por parte del ClassLoader de Spring Boot.

2.23.3. Tipos de archivos .class generados

Los .class pueden corresponder a:

-Controladores (controller/*.class)

Ej.:

- AdminController.class
- AuthController.class
- CentrosController.class
- PuntoDeInteresController.class
- VisitasController.class

Rol técnico:

- Exponen endpoints REST.
- Son detectados por el escaneo de componentes de Spring (@RestController).
- El .class contiene las anotaciones procesadas que Spring utiliza en tiempo de ejecución.

-Servicios (service/*.class)

Ej.:

- CentroService.class
- CentroServiceImpl.class
- UsuarioServiceImpl.class
- VisitaVirtualServiceImpl.class

Rol técnico:

- Contienen la lógica de negocio ya compilada.
- Spring carga estos .class como beans mediante anotaciones como @Service.
- Implementan interfaces compiladas (.class de sus interfaces correspondientes).

-Modelos / Entidades (model/*.class)

Ej.:

- Usuario.class
- Centros.class
- PuntoDeInteres.class
- VisitaVirtual.class

Rol técnico:

- Contienen metadatos JPA ya procesados.
- Hibernate los usa para generar tablas y mapear datos.
- Incluyen anotaciones compiladas como @Entity, @Column, @Id, etc.

-Repositorios (repository/*.class)

Ej.:

- UsuarioRepository.class
- VisitaVirtualRepository.class
- PuntoDeInteresRepository.class

Rol técnico:

- Son proxies generados dinámicamente por Spring Data JPA.
- Contienen definiciones de consultas compiladas.
- Hibernate los utiliza como capa de acceso a datos.

-Configuración (config/*.class)

Ej.:

- SecurityConfig.class
- CorsConfiguration.class
- AppRoleConfig.class

Rol técnico:

- Contienen configuraciones que Spring carga mediante reflexión.

- Representan beans que configuran seguridad, CORS, roles, etc.

-Seguridad (security/jwt/*.class)

Ej.:

- JwtUtils.class
- JwtAuthenticationFilter.class
- AuthEntryPointJwt.class

Rol técnico:

- Implementan filtros de Spring Security ya compilados.
- Se ejecutan antes de cada solicitud HTTP.
- Generan, validan y decodifican tokens JWT.

-DTOs (dto/*.class)

Ej.:

- LoginRequest.class
- RegisterRequest.class
- PuntoDeInteresDTO.class

Rol técnico:

- Estructuras compiladas usadas para transporte de datos en requests/responses.

Excepciones personalizadas

Ej.:

- `ResourceNotFoundException.class`

Rol técnico:

- Contienen la lógica de excepciones de negocio ya compilada.
- Spring las detecta y maneja vía controladores.

2.23.4 ¿Por qué son importantes los .class en Spring Boot?

1. Permiten que el proyecto se ejecute en la JVM

El servidor integrado Tomcat carga los .class y monta el backend.

2. Spring usa reflexión sobre los .class

Permite:

- detectar anotaciones
- inyectar dependencias
- mapear entidades JPA
- construir el contexto de seguridad

3. Son portables

Solo necesitas los .class para ejecutar la app en cualquier servidor donde haya Java.

4. Son los que terminan dentro del .jar de despliegue

El JAR final contiene los .class y recursos necesarios.

2.23.5 Archivo .class principal

-VisitasvirtualesBackendApplication.class

Este es el **punto de entrada de la aplicación**, compilado desde:

VisitasvirtualesBackendApplication.java

Función técnica:

- Arranca el contexto de Spring Boot (SpringApplication.run)
- Carga todos los beans
- Configura las rutas, seguridad, JPA y CORS
- Inicia Tomcat embebido

2.23.6 Estructura .class

```
├─ target/
|  | ── classes/
|  |   | ── com/ies/tour/visitasvirtuales_backend/
|  |   |   | ── config/
|  |   |   |   | ── AppRoleConfig.class
|  |   |   |   | ── CorsConfiguration.class
```

- | | | | | | | — SecurityBeansConfig.class
- | | | | | | | — SecurityConfig.class
- | | | | | | — controller/
- | | | | | | | — AdminController.class
- | | | | | | | — AuthController.class
- | | | | | | | — CentrosController.class
- | | | | | | | — PuntoDeInteresController.class
- | | | | | | | — Valid.class
- | | | | | | | — VisitasController.class
- | | | | | | — dto/
- | | | | | | | — LoginRequest.class
- | | | | | | | — PuntoDeInteresDTO.class
- | | | | | | | — RegisterRequest.class
- | | | | | | — exception/
- | | | | | | | — ResourceNotFoundException.class
- | | | | | | — model/
- | | | | | | | — Centros.class
- | | | | | | | — PuntoDeInteres.class
- | | | | | | | — RolUsuario.class
- | | | | | | | — Usuario.class
- | | | | | | | — VisitaVirtual.class
- | | | | | | — payload/response/
- | | | | | | | — JwtResponse.class
- | | | | | | — repository/
- | | | | | | | — CentroRepository.class
- | | | | | | | — PuntoDeInteresRepository.class

- | | | | | | | — UsuarioRepository.class
- | | | | | | | — VisitaVirtualRepository.class
- | | | | | | — security/jwt/
- | | | | | | | — AuthEntryPointJwt.class
- | | | | | | | — JwtAuthenticationFilter.class
- | | | | | | | — JwtUtils.class
- | | | | | | — service/
- | | | | | | | — CentroService.class
- | | | | | | | — CentroServiceImpl.class
- | | | | | | | — PuntoDeInteresService.class
- | | | | | | | — PuntoDeInteresServiceImpl.class
- | | | | | | | — UsuarioService.class
- | | | | | | | — UsuarioServiceImpl.class
- | | | | | | | — Usuarioservice.class
- | | | | | | | — VisitaVirtualService.class
- | | | | | | | — VisitaVirtualServiceImpl.class
- | | | | | | — VisitasvirtualesBackendApplication.class
- | | | — application.properties
- | | | — test-classes/com/ies/tour/visitasvirtuales_backend/
- | | | | — VisitasvirtualesBackendApplicationTests.class

2.24 Diagrama UML — Modelo del Backend

(Representado en UML de texto usando notación tipo PlantUML, pero sin requerir herramientas externas)

@startuml

```
class Centros {  
    - idCentros : Integer  
    - nombre : String  
    - emailCentro : String  
    - telefono : Integer  
    - direccion : String  
    --  
    + visitas : Set<VisitaVirtual>  
    + puntoDeInteres : Set<PuntoDeInteres>  
}
```

```
class PuntoDeInteres {  
    - idPdi : Long  
    - nombre : String  
    - contenidoJson : String  
    --  
    + centro : Centros  
    + creador : Usuario  
}
```

```
class Usuario {  
    - idUsuario : Long  
}
```

```
class VisitaVirtual {  
    - idVisita : Long  
}
```

Centros "1" --> "0..*" VisitaVirtual : tiene >

Centros "1" --> "0..*" PuntoDeInteres : contiene >

PuntoDeInteres "1" --> "1" Usuario : creado por >

PuntoDeInteres "1" --> "1" Centros : pertenece a >

@enduml

Explicación del Diagrama

1. Centros

Entidad principal del backend.

Tiene relación con:

- **VisitaVirtual (1-N)**
- **PuntoDeInteres (1-N)**

Esto refleja las anotaciones @OneToMany.

2. PuntoDeInteres

Representa un punto de interés dentro de un centro.

Claves importantes:

- Guarda un **JSON completo** en contenidoJson.
- Tiene relación con:
 - **Centros (N-1)** → cada PDI pertenece a un centro.
 - **Usuario (N-1)** → cada PDI tiene un creador.

3. Usuario

Sabemos que existe y que tiene una relación **uno a muchos** con PuntoDeInteres.

4. VisitaVirtual

Aparece en Centros, por lo que se incluye como entidad dependiente.

3. Documentación de Pruebas y Calidad

Nota: Este apartado se deja vacío para completar en el futuro, cuando se realicen pruebas unitarias, de integración, UI y rendimiento.

Secciones a incluir posteriormente:

Plan de pruebas

- Qué funcionalidades se probaron, cuándo y quién realizó las pruebas.

Casos de prueba / escenarios

- Detalle de pruebas para cada módulo, flujo o función.

Resultados de pruebas

- Resultados de tests unitarios, integración, UI, rendimiento.

Registro de errores

- Incidencias encontradas, resolución y seguimiento.

Validación del cliente

- Acta de aceptación o confirmación de cumplimiento de requisitos.

4. Documentación Legal y de Entrega

Esta sección formaliza la entrega del proyecto y protege tanto al equipo de desarrollo como al cliente.

4.1 Acta de entrega del proyecto

- Proyecto entregado en versión **1.0**.
- Archivos entregados:
 - **visitasvirtuales_frontend/** (HTML, CSS, JS, imágenes)
 - **visitasvirtuales_backend/** (target, src, pom.xml, mvnw, help.md)
- El proyecto se aloja en **Unity** y está controlado mediante repositorio **GitHub** para el código y **Plastic SCM** para la base de datos.

4.2 Licencias de software utilizadas

- Backend: Java, Spring Boot — licencia **Open Source**.
- Frontend: HTML, CSS, JS — libre de derechos.
- **Dependencias externas: se listan en pom.xml** con sus licencias correspondientes.
- Motor de base de datos y Unity: licencias comerciales según instalación.

4.3 Propiedad intelectual y derechos de uso

- Todos los códigos y recursos gráficos son propiedad del equipo desarrollador.
- El cliente obtiene **derechos de uso, despliegue y modificación** según contrato.
- No se permite redistribución fuera del acuerdo de entrega.

4.4 Contratos de mantenimiento o soporte

- Opcional: servicios de mantenimiento del servidor, actualizaciones y soporte.
- Se detallarán en documento aparte si se acuerda un contrato.